INTERNSHIP PROJECT REPORT

DESIGN AND IMPLEMENTATION OF 1010 SEQUENCE DETECTORS USING VERILOG HDL

Submitted by:

POOJA RAJEEVAYYA KABBINAKANTIMATH

JUNE 2026

Abstract

Sequence detectors are important sequential circuits used in digital systems to identify predefined binary patterns from a serial input stream. These circuits are widely used in communication systems, pattern recognition, embedded systems, and digital signal processing applications. Sequence detectors are generally implemented using Finite State Machines (FSMs), which provide efficient state-based control and pattern detection.

In this project, the binary sequence 1010 is detected using four different FSM architectures: Mealy Overlapping, Mealy Non-Overlapping, Moore Overlapping, and Moore Non-Overlapping sequence detectors. The designs are modeled using Verilog HDL and simulated using Vivado Design Suite. The project compares the behavior, state transitions, output characteristics, and hardware requirements of the different sequence detector architectures. Simulation waveforms verify the correct detection of the desired sequence under both overlapping and non-overlapping conditions.

Chapter 1: Introduction

1.1 Introduction

A sequence detector is a sequential digital circuit designed to detect a specific sequence of binary digits appearing in a serial input stream. Sequence detectors are widely used in communication systems, digital controllers, protocol analyzers, and embedded systems where pattern recognition is required.

Finite State Machines (FSMs) are commonly used to implement sequence detectors because they provide an organized method for tracking previously received bits and generating an output when a desired sequence is detected.

The sequence selected for this project is:

1010

The detector generates an output whenever this pattern appears in the incoming data stream.

1.2 Objectives

The objectives of this project are:

- To understand the concept of Finite State Machines.
- To design Mealy and Moore sequence detectors.
- To implement overlapping and non-overlapping detection techniques.
- To write Verilog HDL code for sequence detectors.
- To develop testbenches for functional verification.
- To analyse simulation waveforms and compare detector performance.

1.3 Applications

Sequence detectors are used in:

- Digital communication systems
- Data synchronization circuits
- Embedded systems
- Protocol analysers
- Pattern recognition systems
- Error detection mechanisms
- Serial data receivers

Chapter 2: Finite State Machines

2.1 Introduction to FSM

A Finite State Machine (FSM) is a sequential logic circuit whose operation depends on the present state and input conditions.

An FSM consists of:

- State Register
- Next State Logic
- Output Logic

FSMs are classified into two types:

1. Mealy Machine
2. Moore Machine

2.2 Mealy Machine

A Mealy machine generates output based on both the present state and the current input.

Equation

$$Output = f(State, Input)$$

Advantages

- Faster response
- Fewer states required
- Lower hardware complexity

Disadvantages

- Output may change asynchronously with input
- Possible glitches

2.3 Moore Machine

A Moore machine generates output based only on the current state.

Equation

$$Output = f(State)$$

Advantages

- Stable output
- Easier debugging
- Better synchronization

Disadvantages

- More states required
- One clock cycle delay in output

Chapter 3: Sequence Detector Concepts

3.1 Overlapping Sequence Detection

In overlapping sequence detection, some bits of a previously detected sequence can be reused to detect the next occurrence of the pattern.

Example

Input Stream:

10101010

Detections:

1010
 1010
  1010

Multiple detections are possible because bits are reused.

3.2 Non-Overlapping Sequence Detection

In non-overlapping sequence detection, once a sequence is detected, the FSM returns to its initial state and begins searching again.

Example

Input Stream:

10101010

Detection:

1010

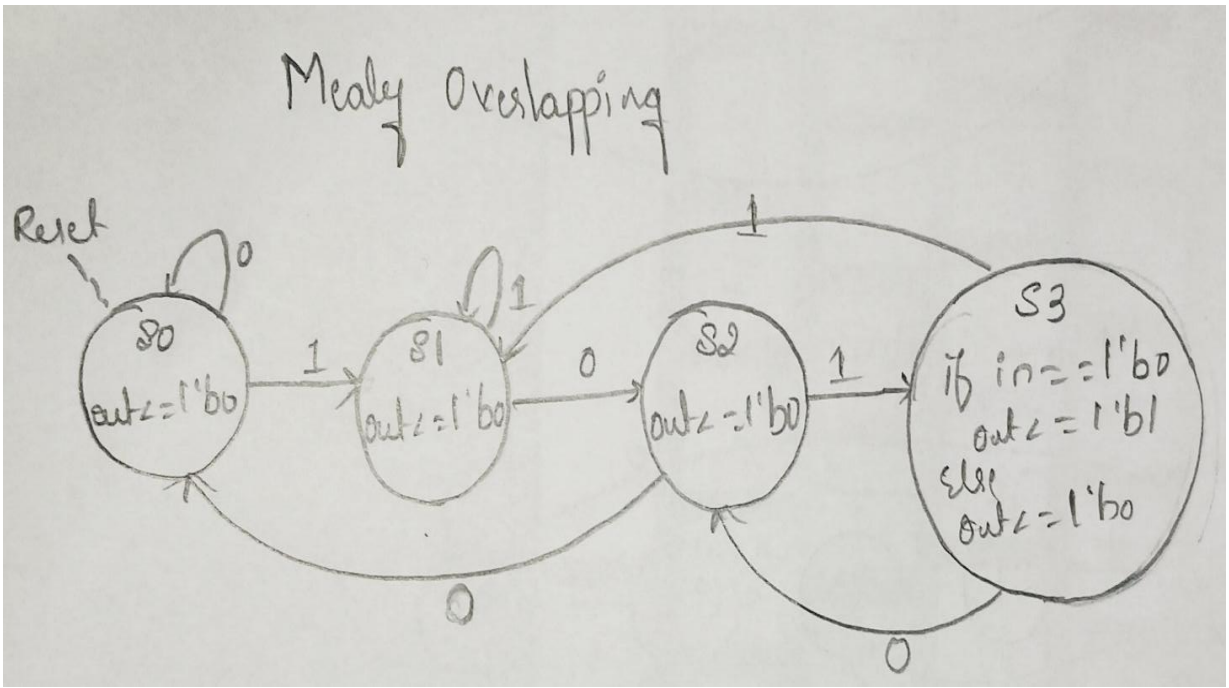
The previously detected bits are not reused.

Chapter 4: Mealy Overlapping Sequence Detector

4.1 Design Description

The Mealy overlapping sequence detector detects the sequence 1010 and allows overlapping occurrences. The output is generated immediately when the last bit of the sequence is received.

4.2 State Diagram



4.3 State Transition Table

Present State	Input (din)	Next State	Output (dout)
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	1
S3	1	S1	0

4.4 Verilog Code

```
`timescale 1ns / 1ps
module mealy_1010_overlap(
    input clk,
    input reset,
    input din,
    output reg dout
);
parameter S0 = 2'b00,
          S1 = 2'b01,
          S2 = 2'b10,
          S3 = 2'b11;
reg [1:0] state, next_state;
// State Register
always @(posedge clk or posedge reset)
begin
    if(reset)
        state <= S0;
```

```
    else
        state <= next_state;
end
// Next State Logic
always @(*)
begin
    case(state)
        S0:
            if(din)
                next_state = S1;
            else
                next_state = S0;
        S1:
            if(din)
                next_state = S1;
            else
                next_state = S2;
        S2:
            if(din)
                next_state = S3;
            else
                next_state = S0;
        S3:
            if(din)
                next_state = S1;
            else
                next_state = S2; // Overlapping
        default:
            next_state = S0;

    endcase
end
```

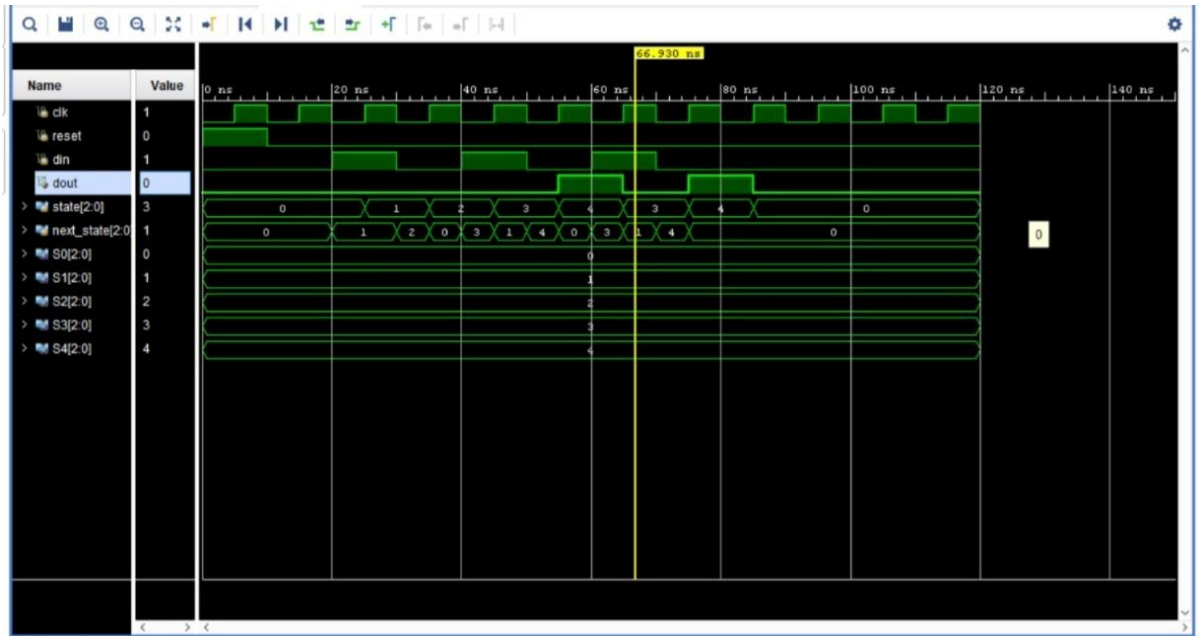
```
// Output Logic (Mealy)
always @(*)
begin
    if(state == S3 && din == 1'b0)
        dout = 1'b1;
    else
        dout = 1'b0;
end
endmodule
```

4.5 Testbench

```
`timescale 1ns / 1ps
module mealy_1010_overlap_tb;
reg clk;
reg reset;
reg din;
wire dout;
// DUT
mealy_1010_overlap uut(
    .clk(clk),
    .reset(reset),
    .din(din),
    .dout(dout)
);
// Clock Generation
always #5 clk = ~clk;
initial
begin
    clk = 0;
    reset = 1;
    din = 0;
    // Reset
```

```
    #10 reset = 0;
    // Input Stream = 10101010
    #10 din = 1;
    #10 din = 0;
    #10 din = 1;
    #10 din = 0;   // First detection
    #10 din = 1;
    #10 din = 0;   // Second detection (overlap)
    #10 din = 1;
    #10 din = 0;   // Third detection (overlap)
    #50;
    $stop;
end
endmodule
```

4.6 Simulation Waveform

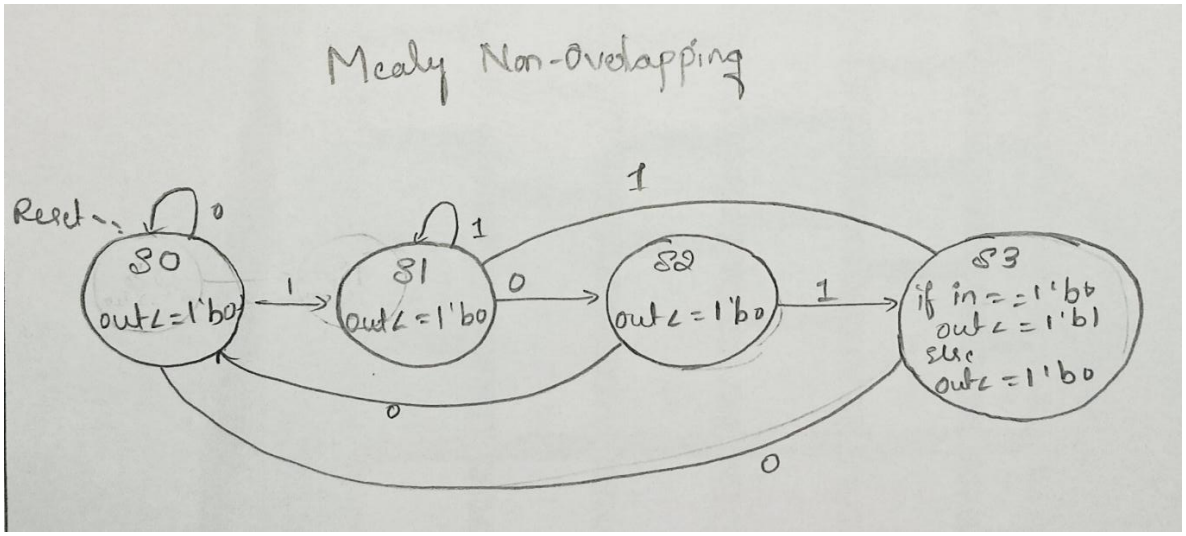


Chapter 5: Mealy Non-Overlapping Sequence Detector

5.1 Design Description

The Mealy non-overlapping sequence detector resets to the initial state after detecting the sequence 1010. Previously detected bits are not reused.

5.2 State Diagram



5.3 State Transition Table

Present State	Input (din)	Next State	Output (dout)
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S0	1
S3	1	S1	0

5.4 Verilog Code

```
module seq_1010_mealy(
    input clk,
    input rst,
```

```
    input din,
    output reg dout
);
reg [1:0] state;
parameter S0 = 2'b00,  // No match
          S1 = 2'b01,  // Got 1
          S2 = 2'b10,  // Got 10
          S3 = 2'b11;  // Got 101
always @(posedge clk or posedge rst)
begin
    if(rst)
    begin
        state <= S0;
        dout <= 0;
    end
    else
    begin
        dout <= 0;
        case(state)
            S0:
            begin
                if(din)
                    state <= S1;
                else
                    state <= S0;
            end
            S1:
            begin
                if(din)
                    state <= S1;
```

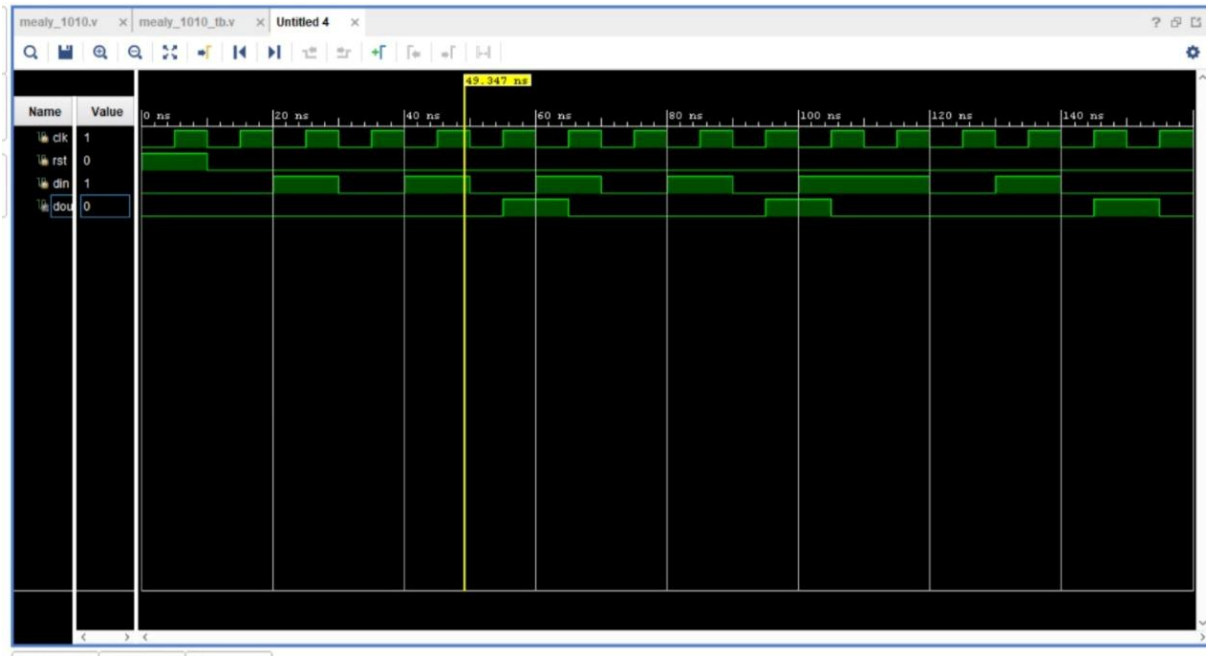
```
            else
                state <= S2;
            end

            S2:
            begin
                if(din)
                    state <= S3;
                else
                    state <= S0;
            end

            S3:
            begin
                if(din)
                begin
                    state <= S1;
                end
                else
                begin
                    dout <= 1;      // 1010 detected
                    state <= S0;    // Non-overlapping
                end
            end

        endcase
    end
end

endmodule
```

5.5 Testbench

```
`timescale 1ns/1ps

module seq_1010_tb;

reg clk;
reg rst;
reg din;
wire dout;

seq_1010_mealy uut (
    .clk(clk),
    .rst(rst),
    .din(din),
    .dout(dout)
);

always #5 clk = ~clk;

initial
begin
    clk = 0;
    rst = 1;
    din = 0;

    #10 rst = 0;

    // Input sequence:
    // 1 0 1 0 -> detect
    // 1 0 1 0 -> detect
```

```
    // 1 1 0 1 0 -> detect
    #10 din = 1;
    #10 din = 0;
    #10 din = 1;
    #10 din = 0;
    #10 din = 1;
    #10 din = 0;
    #10 din = 1;
    #10 din = 0;
    #10 din = 1;
    #10 din = 1;
    #10 din = 0;
    #10 din = 1;
    #10 din = 0;
    #20 $finish;
end
endmodule
```

5.6 Simulation Waveform

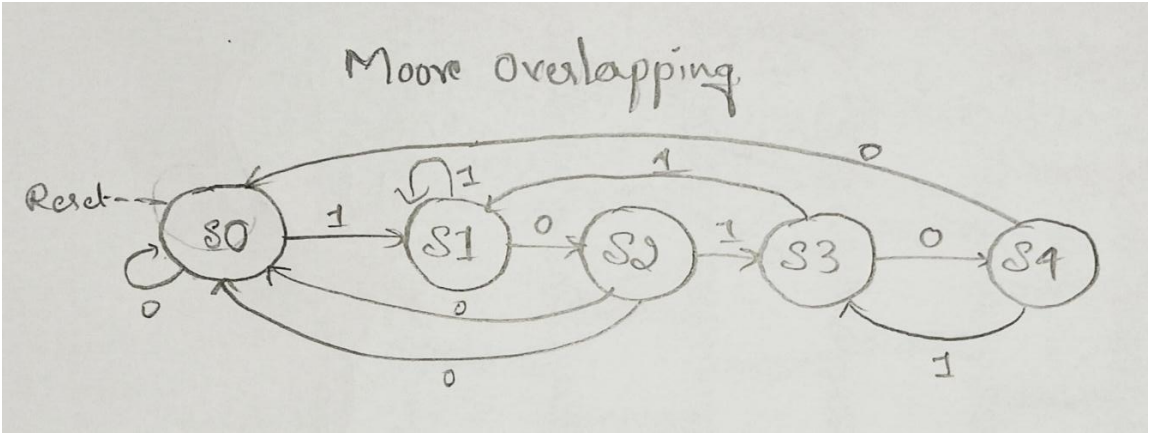


Chapter 6: Moore Overlapping Sequence Detector

6.1 Design Description

The Moore overlapping sequence detector generates output only when the FSM enters the detection state. Overlapping sequence detection is supported.

6.2 State Diagram



6.3 State Transition Table

Present State	Input	Next State	Output
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S4	0
S3	1	S1	0
S4	0	S2	1
S4	1	S1	1

6.4 Verilog Code

```
module moore_1010_overlap(
    input clk,
    input rst,
    input din,
    output reg dout
);
parameter S0=3'b000,
          S1=3'b001,
          S2=3'b010,
          S3=3'b011,
          S4=3'b100;

reg [2:0] state,next_state;
```

```
// State Register
always @(posedge clk or posedge rst)
begin
   if(rst)
      state <= S0;
   else
      state <= next_state;
end

// Next State Logic
always @(*)
begin
   case(state)

   S0: next_state=(din)?S1:S0;

   S1: next_state=(din)?S1:S2;

   S2: next_state=(din)?S3:S0;

   S3: next_state=(din)?S1:S4;

   S4: next_state=(din)?S1:S2; // Overlapping

   default: next_state=S0;

   endcase
end

// Output Logic
```

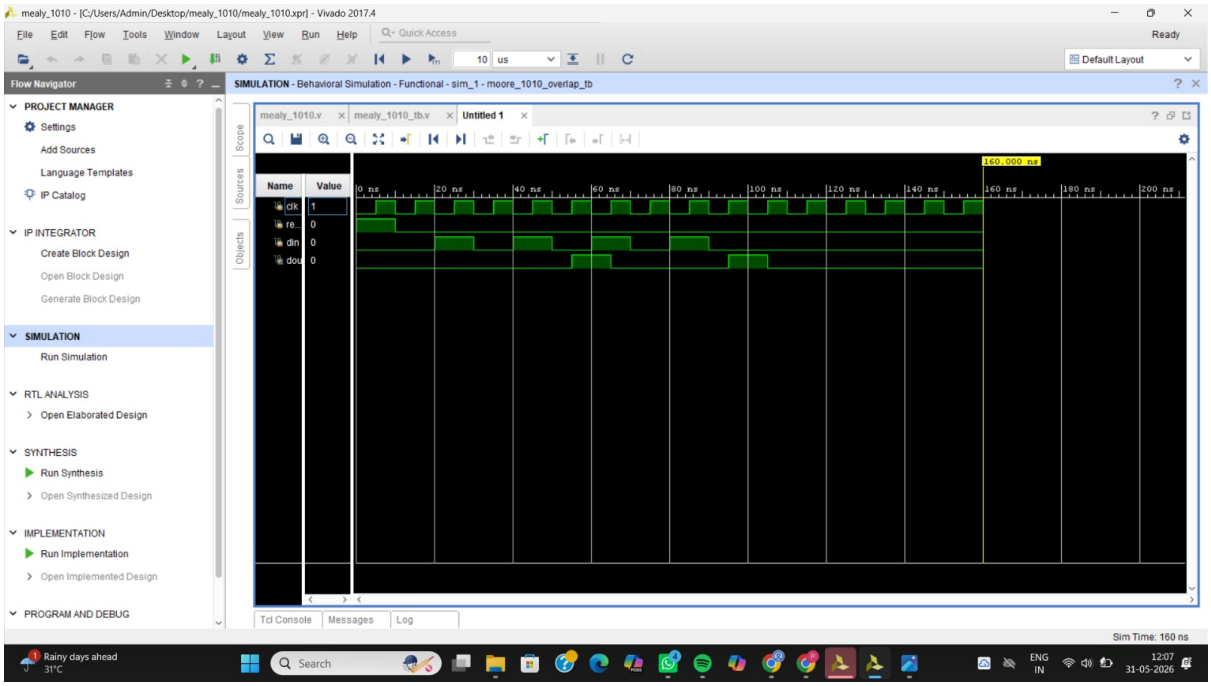
```
always @(*)
begin
    dout=(state==S4);
end
endmodule
```

6.5 Testbench

```
`timescale 1ns/1ps
module moore_1010_overlap_tb;
reg clk,rst,din;
wire dout;
moore_1010_overlap uut(
    .clk(clk),
    .rst(rst),
    .din(din),
    .dout(dout)
);
always #5 clk=~clk;
initial
begin
    clk=0;
    rst=1;
    din=0;
    #10 rst=0;
    //10101010
    #10 din=1;
    #10 din=0;
    #10 din=1;
    #10 din=0;
    #10 din=1;
    #10 din=0;
```

```
    #10 din=1;
    #10 din=0;
    #50 $finish;
end
endmodule
```

6.6 Simulation Waveform

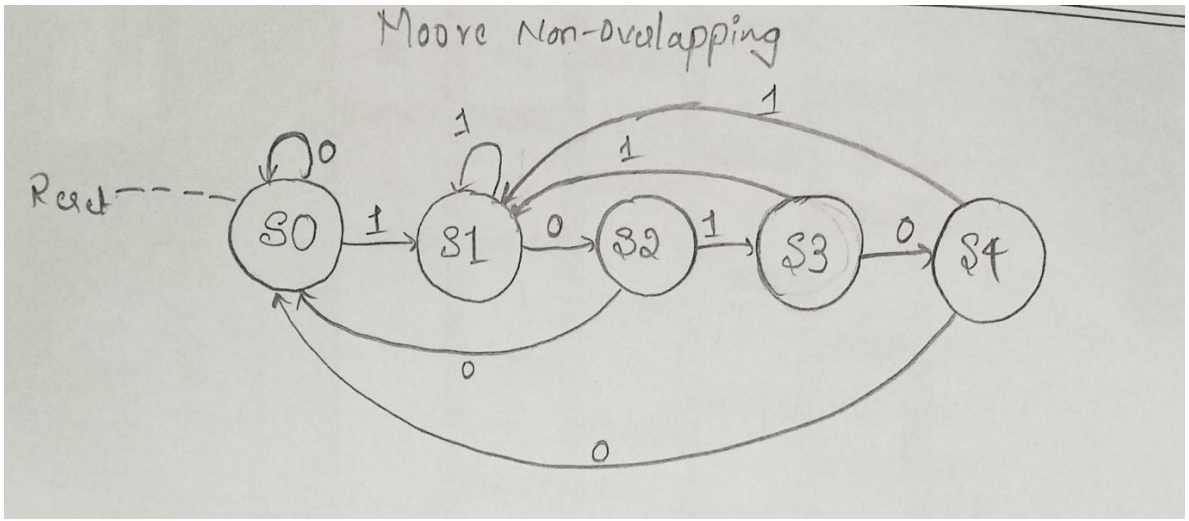


Chapter 7: Moore Non-Overlapping Sequence Detector

7.1 Design Description

The Moore non-overlapping sequence detector generates output after entering the detection state and returns to the initial state after detection.

7.2 State Diagram



7.3 State Transition Table

Present State	Input	Next State	Output
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S4	0
S3	1	S1	0
S4	X	S0	1

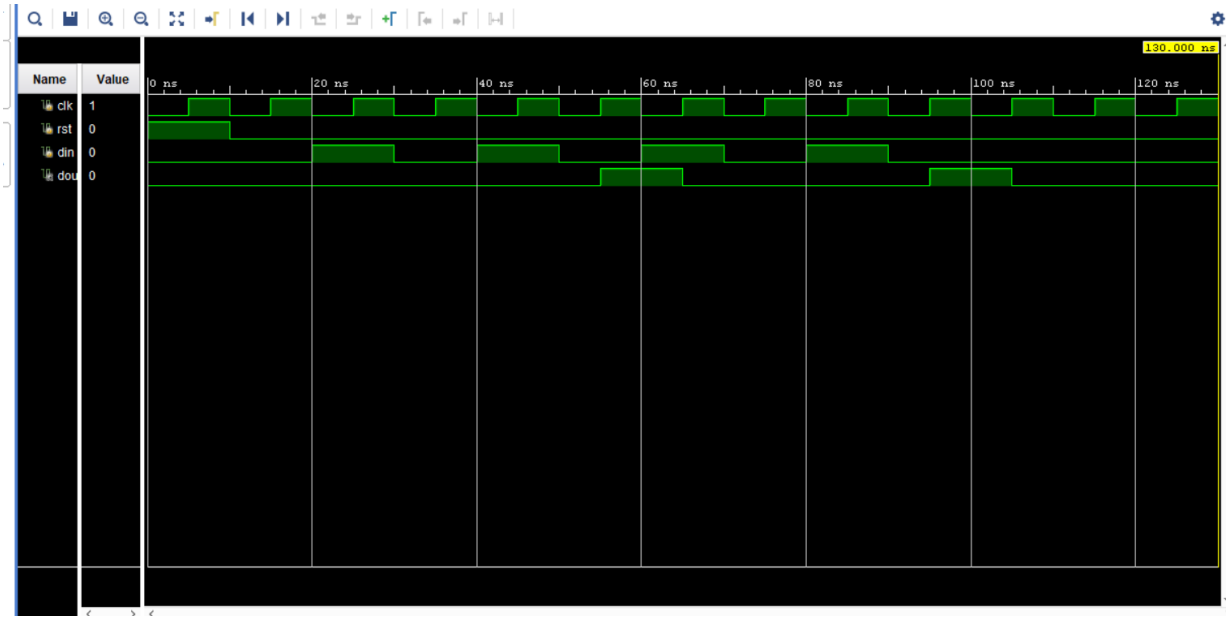
7.4 Verilog Code

```
`timescale 1ns/1ps
module moore_1010_nonoverlap(
    input clk,
    input rst,
    input din,
    output reg dout
```

```
);
parameter S0 = 3'b000,
          S1 = 3'b001,
          S2 = 3'b010,
          S3 = 3'b011,
          S4 = 3'b100;
reg [2:0] state, next_state;
// State Register
always @(posedge clk or posedge rst)
begin
   if(rst)
      state <= S0;
   else
      state <= next_state;
end
// Next State Logic
always @(*)
begin
   case(state)
      S0: next_state = (din) ? S1 : S0;
      S1: next_state = (din) ? S1 : S2;
      S2: next_state = (din) ? S3 : S0;
      S3: next_state = (din) ? S1 : S4;
 // Non-overlapping
      S4: next_state = (din) ? S1 : S0;
      default: next_state = S0;
   endcase
end
// Output Logic
always @(*)
```

```
begin
    if(state == S4)
        dout = 1'b1;
    else
        dout = 1'b0;
end
endmodule
endmodule
```

7.6 Simulation Waveform



Chapter 8: Comparison of Sequence Detectors

Feature	Mealy Overlapping	Mealy Non-Overlapping	Moore Overlapping	Moore Non-Overlapping
Number of States	4	4	5	5
Output Delay	No	No	Yes	Yes
Hardware Complexity	Low	Low	Medium	Medium
Response Speed	Fast	Fast	Moderate	Moderate
Output Stability	Medium	Medium	High	High
Overlap Support	Yes	No	Yes	No

Chapter 9: Results and Discussion

Simulation results obtained from Vivado Design Suite confirm the successful detection of the sequence 1010 for all four implementations. The Mealy machines produce outputs immediately when the sequence is detected, while Moore machines generate outputs after entering the detection state, resulting in a one-clock-cycle delay.

Overlapping sequence detectors successfully identify multiple occurrences of the sequence using shared bits, whereas non-overlapping detectors restart the search after each successful detection. The simulation waveforms validate the correct behaviour of all FSM architectures.

Chapter 10: Conclusion

The design and implementation of Mealy and Moore sequence detectors for the sequence 1010 were successfully completed using Verilog HDL. Both overlapping and non-overlapping detection techniques were implemented and verified using simulation waveforms.

The project demonstrates the practical application of Finite State Machines in digital system design. It also highlights the differences between Mealy and Moore architectures in terms of response time, hardware complexity, output stability, and state requirements.

Chapter 11: Future Scope

Future enhancements may include:

- Detection of longer binary sequences.
- Programmable sequence detectors.
- FPGA implementation and hardware testing.
- Low-power FSM optimization techniques.
- Real-time communication protocol analysis.
- Reconfigurable pattern detection systems.